

IBM - NALAIYA THIRAN

Web Phishing Detection using Machine Learning

Team ID: PNT2022TMID02425

Done By:

Raya Rohith Yadav

Raghul. C

Sakthivel. P

Prasanna Srinivas

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1.	INTRODUCTION	
1.1	Project Overview	4
1.2	Purpose	4
2.	LITERATURE SURVEY	
2.1	Existing problem	5
2.2	References	5
2.3	Problem Statement Definition	7
3.	IDEATION & PROPOSED SOLUTION	
3.1	Empathy Map Creation	8
3.2	Ideation & Brainstorming	8
3.3	Proposed Solution	9
3.4	Problem Solution fit	12
4.	REQUIREMENT ANALYSIS	
4.1	Functional requirement	13
4.2	Non-Functional requirements	14
5.	PROJECT DESIGN	
5.1	Data Flow Diagrams	15
5.2	Solution & Technical Architecture	15
5.3	User Stories	16

6.	PROJECT PLANNING & SCHEDULING	
6.1	Sprint Planning & Estimation	18
6.2	Sprint Delivery Schedule	19
6.3	Reports from JIRA	19
7.	CODING & SOLUTIONING	
7.1	Feature 1	20
7.2	Feature 2	39
8.	TESTING	
8.1	Test Cases	56
8.2	User Acceptance Testing	56
9.	RESULTS	
9.1	Performance Metrics	58
10.	ADVANTAGES & DISADVANTAGES	61
11.	CONCLUSION	62
12.	FUTURE SCOPE	62
13.	APPENDIX	63
13.1	Source code	
13.2	GitHub and Project demo link	

1. INTRODUCTION

1.1 Project overview:

Phishing is a cybercrime in which a target user is contacted by different communicative means by someone posing as a legitimate institution to lure individuals into ultimate identity theft and financial loss.

It is a non-ethical method comprising both social engineering and technical tricks to capture user's information and sensitive credentials like personally identifiable information, banking and credit card details, and passwords. Some of the social engineering techniques use spam mails, pretending as a legitimate company or organization and message of urgency to commit an act. The information is then used to access target's personal data and credentials and trying to behave as a genuine user themselves.

The attacker can commit this act by different phishing methodologies as each methodology aims in different aim of retrieving the users information. Well there are different ways to handle and avoid getting phished but they cannot be proposed in a practical manner because of the sophisticated transformation of taking cybercrimes into new paths and alternates increases the attack rate.

1.2 Purpose:

Roughly 15 billion spam emails make their way across the internet every day, and are liable to permit malicious phishing attack emails to slip through. In 2021, 83% of organizations reported experiencing phishing attacks. Phishing campaigns accounts for losses of \$17,700 per minute

With the booming of internet transactions, cybercrimes rapidly increased and with anonymity presented by the internet, Hackers attempt to trap the end-users through various forms such as phishing, SQL injection, malware, man-in-the-middle, domain name system tunnelling, ransomware, web trojan, and so on.

To overcome the drawbacks of blacklist and heuristics based method, many security researchers now focused on machine learning techniques. Machine learning technology consists of a many algorithms which requires past data to make a decision or prediction on future data. Using this technique, algorithm will analyse various blacklisted and legitimate URLs and their features to accurately detect the phishing websites including zero- hour phishing websites.

2. LITERATURE SURVEY

2.1 Existing problem:

Phishing is a major threat to all users that parse and access information through internet and is difficult to trace or defend against since it does not present itself as obvious malicious in nature. In today's society, everything is put online and the safety of personal credentials is at risk. Phishing can be seen as one of the oldest and easiest ways of stealing information from people and it is used for obtaining a wide range of personal details. It also has a fairly simple approach – send an email, email sends victim to a site, site steals information. It is evident from the fact that though users get alert on some cheap crafted phished sites, attackers approach innovative and descriptive way to achieve in committing fraudulent activities.

2.2 References:

- **Rao et al** proposed a novel classification approach that use heuristic based feature extraction approach. In this, they have classified extracted features into three categories such as URL Obfuscation features, Third-Party-based features, Hyperlink-based features. Moreover, proposed technique gives 99.55% accuracy. Drawback of this is that as this model uses third-party features, classification of website dependent on speed of third-party services. Also this model is purely depends on the quality and quantity of the training set and Broken links feature extraction has a limitation of more execution time for the websites with more number of links.

Source: Routh Srinivasa Rao¹ , Alwyn Roshan Pais : Detection of phishing websites using an efficient feature-based machine learning framework :In Springer 2018.

- **Nepali and Wang** proposed a novel approach to detect malicious URLs using only visible features from social networks.

Source: R. K. Nepali and Y. Wang, Y., “You look suspicious!! Leveraging visible attributes to classify malicious short URLs on twitter”, 2016 49th Hawaii International Conference on System Sciences (HICSS). IEEE, pp. 2648–2655, 2016.

- **Patil** provided a brief overview of various forms of web-page attacks in their survey on malicious webpages detection techniques.

Source: D. R. Patil and J. Patil, J., “Survey on malicious web pages detection techniques”, International Journal of u-and e-Service, Science and Technology, vol. 8, no. 5, pp. 195–206, 2015.

- **Abdelhamid et al.** built a system for detecting phishing URLs called Multi-label Classifier based on Associative Classification (MCAC). They used sixteen features and classified URLs into three classes: phishing, legitimate, and suspicious. The MCAC is a rule-based algorithm where multiple label rules are extracted from the phishing data set.

Source: N. Abdelhamid, A. Ayesh, F. Thabtah, “Phishing Detection based Associative Classification”, Data Mining. Expert Systems with Applications (ESWA), vol. 41, pp 5948-5959, 2014.

- **Hadi et al.** used the Fast-Associative Classification Algorithm (FACA) for classifying phishing URLs. FACA works by discovering all frequent rule item sets and building a model for classification. They investigated a data set consisting of 11,055 websites with two classes, legitimate and phishing. The data set contained thirty features. They used the minimum support and the minimum confidence threshold values as two percent and fifty percent, respectively.

Source: W. Hadi, F. Aburrub, and S, Alhawari, “A new fast associative classification algorithm for detecting phishing websites”, Applied Soft Computing vol. 48, pp 729-734, 2016.

- **Kuyama et al** proposed a method for identifying the Command and Control server (C&C server) by using supervised learning and features points obtained from WHOIS and DNS information. They evaluated domain names and email addresses from the WHOIS as input values for machine learning.

Source: M. Kuyama, Y. Kakizaki, and R. Sasaki, “Method for detecting a malicious domain by using Whois and DNS features”, The Third International Conference on Digital Security and Forensics (DigitalSec2016), p. 74, 2016.

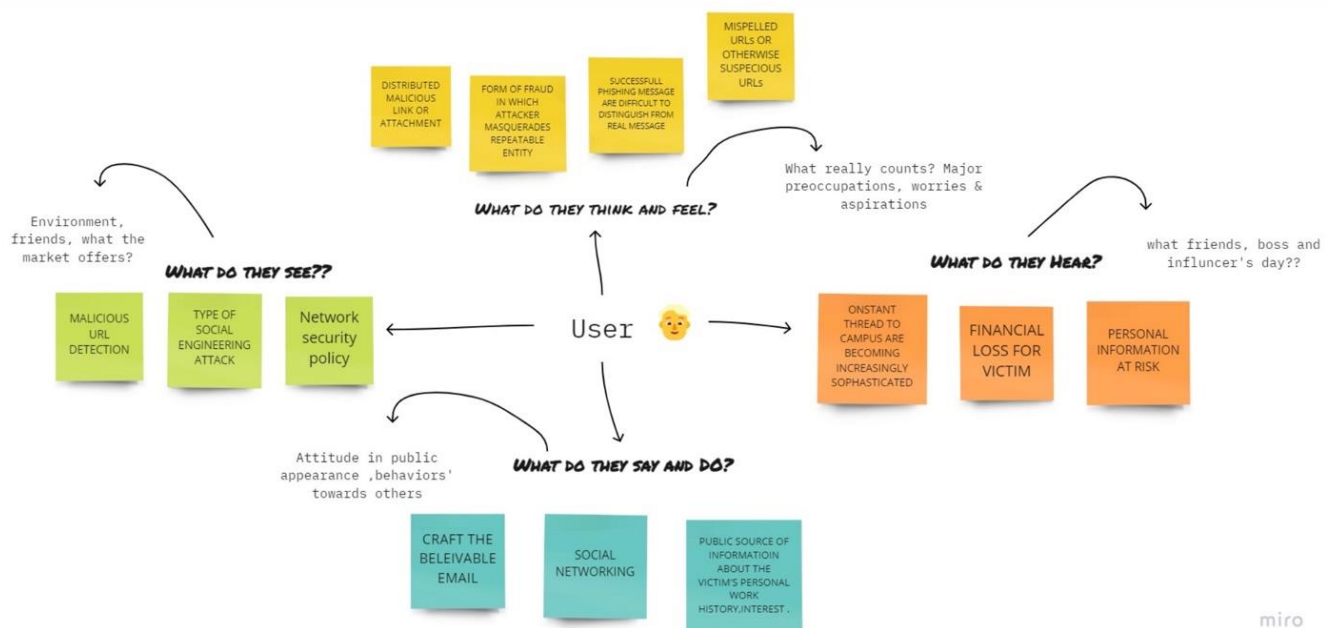
2.3 Problem Statement Definition:

Phishing is a non-ethical method comprising both social engineering and technical tricks to capture user's information and sensitive credentials like financial credentials. Some of the social engineering techniques use spam mails, pretending as a legitimate company or organization, that are specially designed to forefront users to knock-off websites that manoeuvre recipients to fall into the trap which steal financial credentials like user-ids and passwords. Technical intrigue methods install malicious software onto the systems, to capture the data directly, often using systems to intercept users online account user names and passwords. There are multiple websites who ask user to provide sensitive data such as username, password or credit card details etc. often for malicious reasons. Over the years Internet has dominated the world by dragging half of the world's population exponentially into the cyber world. With the booming of internet transactions, cybercrimes rapidly increased and with anonymity presented by the internet, Hackers attempt to trap the end-users through various forms such as phishing, SQL injection, malware, man-in-the-middle, domain name system tunnelling, ransomware, web trojan, and so on. Among all these attacks, phishing reports to be the most deceiving attack.

3.IDEATION AND PROPOSED SOLUTION

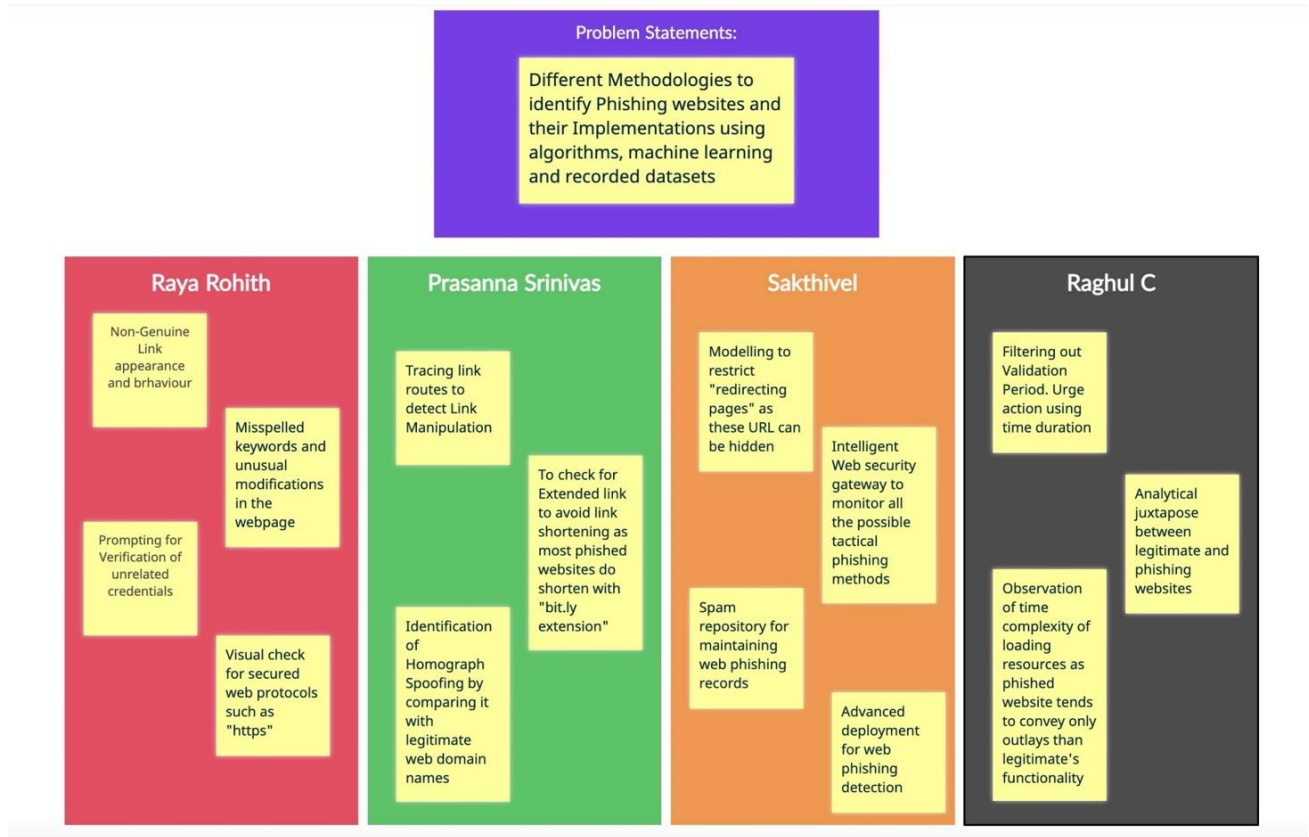
3.1 Empathy Map Canvas:

Empathy mapping is used to analyze and get deeper into the insight tools that's been offered to uptake the task. Relevant contents of specific topics are discussed to make an out graph of the project.



3.2 Ideation and Brainstorming:

The Ideation Phase shows the problem statement along with the specific task, each member of the team is allocated with. This along with the combination of brainstorming allows the members to bifurcate the responsibilities which may lead into better understanding of the problem cases. Allocated task results in specializing in the particular domain lends deeper insights in solving the test cases.



3.3 Proposed Solution:

S.No.	Parameter	Description
1.	Problem Statement	Phishing is a non-ethical method comprising both social engineering and technical tricks to capture user's information and sensitive credentials like financial credentials. Some of the social engineering techniques use spam mails, pretending as a legitimate company or organization, that are specially designed to forefront users to knock-off websites that manoeuvre recipients to fall into the trap which steal financial credentials like user-ids and passwords. Technical intrigue methods install malicious software onto the systems, to capture the data directly, often using systems to intercept users online account user names and passwords. There are multiple websites who ask user to provide sensitive data such as username, password or credit

		card details etc. often for malicious reasons. Over the years Internet has dominated the world by dragging half of the world's population exponentially into the cyber world. With the booming of internet transactions, cybercrimes rapidly increased and with anonymity presented by the internet, Hackers attempt to trap the end-users through various forms such as phishing, SQL injection, malware, man-in-the-middle, domain name system tunnelling, ransomware, web trojan, and so on. Among all these attacks, phishing reports to be the most deceiving attack.
2.	Idea /Solution Description	The model we proposed is capable of detecting and predicting e-banking phishing websites effectively, using machine learning classification algorithms. The e-banking phishing website can be detected by considering some of the key factors like the domain identity, URL, security & encryption criteria in the phishing detection rate. In order to get rid of phishing attacks one needs to change the passwords continuously and use anti-phishing websites to prevent from attacks. And never share you personal / financial details over the internet.
3.	Novelty Uniqueness	Machine learning technology consists of many algorithms which requires past data to make a decision or prediction of future data. Using this technique, algorithm will analyse various backlisted and legitimate URL's and their features to accurately detect the phishing websites including zero-hour phishing websites. For detection of phishing websites we use a set of features like the domain identity, length of the URL and many more and also to make this software computational free we are using SAAS and further planning to make our site to be available in an extension format with UI/UX designing.
4.	Social Impact / Customer Satisfaction	Phishing websites has a lot of effects in terms of property loss, data loss, privacy loss, damage of reputation and disruption of operational activities. Some of the companies like JP morgen, Sony

		pictures effected by data breach due to phishing . • JPMorgan Chase experienced , one of the most significant phishing breaches in history. In 2014, the company announced that the contact information for 76 million households and seven million businesses were compromised in the massive attack. • Using phishing and spear phishing emails, which contained malware, the attackers gained access to Sony’s network and performed months of covert reconnaissance. This attack lead to a cost of \$100 million. Customer satisfaction: By using our web phishing detection website the user can check their websites by copy and paste the phishing URL. After knowing the result they can be completely safe from above mentioned impacts.
5.	Business Model (Revenue Model)	As long as phishing websites continue to operate/emerge, many more people and companies will suffer privacy leaks and data breaches or financial loses/intellectual losses. However, the existing phishing detection method do not fully analyse the features of phishing and the performance and efficiency of the models only apply to certain limited datasets and up to a set of 10 features. further need to be improved to be applied to the real web environment.
6.	Scalability of the Solution	There are vast no. of algorithms that has been in existence which provides wide range of features to identify and analysis web phishing. The concept of phishing has become quite common now such that it can be adapted to any logical approach conducted. This tool we proposed gives high performance and optimization. All sorts of web application and ease of preventing users from scam.

3.4 Problem Solution Fit:

Web Phishing Solution Fit:

1. CUSTOMER SEGMENT Users that uses internet for sharing and accessing resources Eg: Users perform bank transaction, subscribing on a scheme	7. CUSTOMER CONSTRAINTS Phishing attacks tend to behave and function like legitimate sites fooling users. Not aware and knowledge on how phishing attacks work.	4. AVAILABLE SOLUTIONS Manual self-analysis by verifying upon web address's anonymous characteristics. Cross verifying link with phishing database
2. PROBLEMS Confidential Threat, Bankrupt, Data Leaks, System Malfunctioning Prior motive is to concern on end-user's privacy	8. PROBLEM ROOT CAUSE New methods adapted by attackers. Algorithms low efficacy rates. Ambiguous access to credentials	5. BEHAVIOUR Using a custom extension that analysis the current link. Users access the extension which provide the results.
3. TRIGGERS In form of Alerts by an entity Urge / Temptation to commit a task. Awareness of such attacks that could steal personal credentials without any indication.	9. EMOTIONS: BEFORE /AFTER Before: Fear of Insecurity, Subject to vulnerability After: Confidence in accessing websites, Relief of privacy maintain	6. YOUR SOLUTION Detecting Phishing websites by Machine Learning & Classification Algorithm. Use of Pre-defined blacklisted website database

4. REQUIREMENT ANALYSIS

4.1 Functional Requirement:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Registration	Registration through Forms.
FR-2	User Confirmation	The Registration is confirmed via Email.
FR-3	User Authentication	Authentication can be done with password or 2step-verification.
FR-4	User input	User need to download the extension and then inputs an URL (Uniform Resource Locator) in necessary field to check its validation.
FR-5	Website Evaluation	Model evaluates the website using Blacklist and Whitelist approach
FR-6	Extraction and Prediction	It retrieves features based on heuristics and visual similarities. The URL is predicted by the model using Machine Learning methods such as Logistic Regression and KNN.
FR-7	Real Time monitoring	The use of Extension plugin should provide a warning pop-up when they visit a website that is phished. Extension plugin will have the capability to also detect latest and new phishing websites
FR-8	Authentication	Authentication assures secure site, secure processes and enterprise information security.

4.2 Non - Functional Requirements:

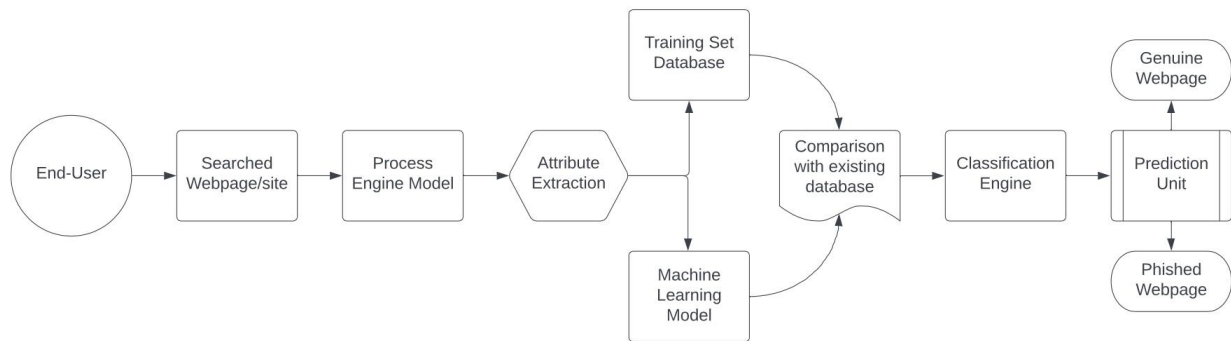
Following are the non-functional requirements of the proposed solution.

FR No.	Non-Functional Requirement	Description
NFR-1	Usability	Usability is commonly considered to be the enemy of security. In general, being secure means taking extra steps to avoid falling for different attacks. This is especially true of phishing where the best ways to prevent most phishing attacks are commonly known, but cybersecurity guidance is rarely followed.
NFR-2	Security	It guarantees any data within the system or its components will be safe from malware threats or unauthorised access. If you wish to prevent unauthorised access to the admin panel, describe the login flow and different user roles as system behaviour.
NFR-3	Reliability	It specifies the likelihood that the system or its component will operate without failure for a specified amount of time under prescribed conditions.
NFR-4	Performance	It is concerned with a measurement of the system's reaction time under various load circumstance.
NFR-5	Availability	Represents likelihood of user access. While it can be represented as an expected proportion of successful requests, it can also be defined as a percentage of time the system is operational.
NFR-6	Scalability	It has access to the highest workloads that allows system to satisfy the performance criteria. Two techniques to enable the system to grow as workloads increase: Vertical & Horizontal

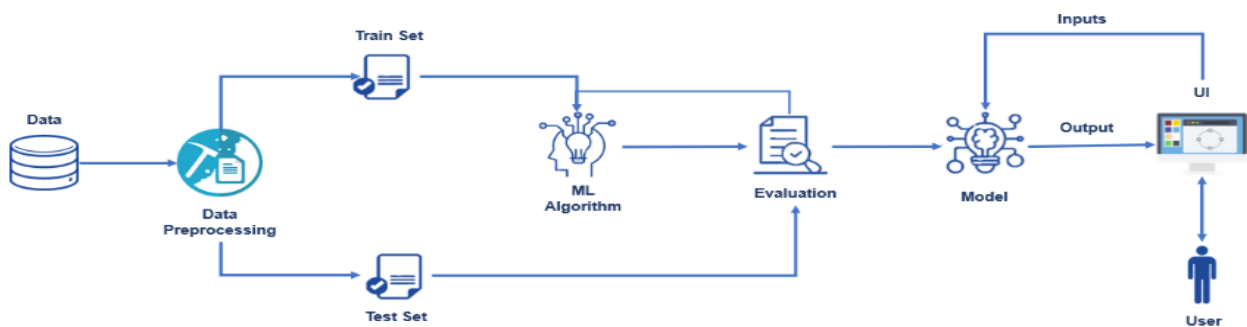
5.PROJECT DESIGN

5.1 Data Flow Diagram:

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It shows how data enters and leaves the system, what changes the information, and where data is stored.



5.2 Solution and Technical Architecture:



5.3 User Stories:

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
Customer (Web User)	Registration	USN-1	As a user, I can register by crediting with login credentials like email address, password, and confirming my password.	I can access my account / dashboard	High	Sprint-1
		USN-2	As a user, I get acknowledged by a token of confirmation via email.	I can get confirmation link via email	High	Sprint-1
		USN-3	As a user, I can double verify by using a double factor authentication.	I can add more security keys to maintain privacy	Medium	Sprint-2
		USN-4	As a user, I can finally login/ access a webpage that does not re-direct to other webpages.	I can cross-verify that stating it could be legitimate	Low	Sprint-1
	Login	USN-5	As a user, I can log into the webpages by email address & password.	I can gain account resources and data	Low	Sprint-1
Customer (Web user)	Input	USN-6	As a user, I share the website's Unified resource locator to the Modal	I can upload the input to the modal	Low	Sprint-2
Customer Care Executive	Attribute Extractor	USN-7	Attribute Extractor and Process Engine works together to	I can verify by comparison	High	Sprint-1

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
			compare with Training and Testing Database to verify matching instances.	algorithm with existing database		
Administrator	Algorithm	USN-8	Machine learning and algorithms are used to process apart from simple approach such as visual identification.		High	Sprint-1
	Result	USN-9	The prediction unit confirms and finalise the resulting input webpage is genuine or not.	I can get the final verdict from the engine	High	Sprint-2

6.PROJECT PLANNING & SCHEDULING

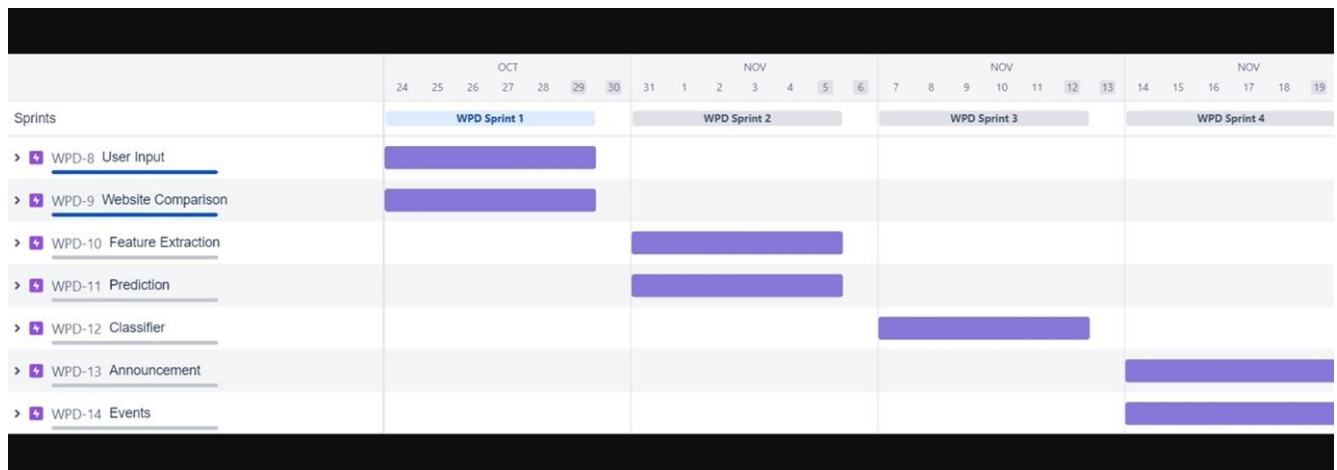
6.1 Sprint Planning and Estimation:

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	User input	USN-1	User inputs an URL in the required field to check its validation.	1	Medium	Raya Rohit.R
Sprint-1	Website Comparison	USN-2	Model compares the websites using Blacklist and Whitelist approach.	1	High	Sakthivel.P
Sprint-2	Feature Extraction	USN-3	Post comparison, if none found then it extract feature using heuristic and visual similarity.	2	High	Raghul.C Prasannasrinivas.R
Sprint-2	Prediction	USN-4	Model predicts the URL using Machine learning algorithms such as logistic Regression, KNN.	1	Medium	Raya Rohit.R
Sprint-3	Classifier	USN-5	Model sends all the output to the classifier and produces the final result.	1	Medium	Raghul.C
Sprint-4	Announcement	USN-6	Model then displays whether the website is legal site or a phishing site.	1	High	Sakthivel.P
Sprint-4	Events	USN-7	This model must meet the capability of retrieving and being accurate.	1	High	Prasannasrinivas.R

6.2 Sprint Delivery Schedule:

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1	20	6 Days	24 Oct 2022	29 Oct 2022	20	29 Oct 2022
Sprint-2	20	6 Days	31 Oct 2022	05 Nov 2022	20	05 Nov 2022
Sprint-3	20	6 Days	07 Nov 2022	12 Nov 2022	20	12 Nov 2022
Sprint-4	20	6 Days	14 Nov 2022	19 Nov 2022	20	19 Nov 2022

6.3 Reports from JIRA:



7.CODING & SOLUTIONING

7.1 Feature 1:

This code contains one such trained algorithm that uses a combination of different algorithms used to identify phished sites using Machine Learning techniques.

Objective

A Phishing website is common social engineering method that mimics trustful uniform locators (URLs) and webpages. The objective of this project is to train machine learning models and deep neural networks on the dataset created to predict phished websites. Both phishing and benign URLs of websites are gathered to form a dataset and from the required URL and website content-based features are extracted. The performance level of each model is measured and compared.

Loading Dataset

The features are extracted and stored in 'csv' file. The working of this can be seen in "Phishing Website Detection_Feature Extraction.ipynb" file. The resulted 'csv' file is uploaded to the jupyter notebook and stored in the dataset.

Importing Packages

By loading the packages, we can manipulate the dataset using the package features and methods.

```

# importing basic packages
import pandas as pd
df = pd.DataFrame()
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

```

Deploying code into IBM Code

This is done to ensure that the file is accessible using the IBM Cloud services .

```

import os, types
import pandas as pd
from botocore.client import Config
import ibm_boto3

def __iter__(self): return 0

# @hidden_cell
# The following code accesses a file in your IBM Cloud Object Storage. It is
# You might want to remove those credentials before you share the notebook.
cos_client = ibm_boto3.client(service_name='s3',
                              ibm_api_key_id='M0hnpznsKh0Lfw8GrFFibWD3tQHqIOZH8Ep3kM3WP1Ca',
                              ibm_auth_endpoint="https://iam.cloud.ibm.com/oidc/token",
                              config=Config(signature_version='oauth'),
                              endpoint_url='https://s3.private.us.cloud-object-storage.appdomain.cloud')

bucket = 'webpishing-donotdelete-pr-wtrfiarwfemmb'
object_key = 'dataset_website.csv'

body = cos_client.get_object(Bucket=bucket, Key=object_key)['Body']
# add missing __iter__ method, so pandas accepts body as file-like object
if not hasattr(body, "__iter__"): body.__iter__ = types.MethodType( __iter__,
                                                                    body)

data0 = pd.read_csv(body)

data0.head()

```

O/P:

index	having_IP	having_IP_Address	URLURL_Length	Shortning_Service	having_At_Symbol	double_slash_redirecting	Prefix_Suffix	having_Sub_Domain	SSLfinal_State	Domain_registration_length	...
0	1		-1	1	1	1	-1	-1	-1	-1	...
1	2		1	1	1	1	-1	0	1	-1	...
2	3		1	0	1	1	-1	-1	-1	-1	...
3	4		1	0	1	1	-1	-1	-1	1	...
4	5		1	0	-1	1	-1	1	1	-1	...
=====											
popUpWidnow	Iframe	age_of_domain	DNSRecord	web_traffic	Page_Rank	Google_Index	Links_pointing_to_page	Statistical_report	Result		
1	1	-1	-1	-1	-1	1	1	-1	-1		
1	1	-1	-1	0	-1	1	1	1	-1		
1	1	1	-1	1	-1	1	0	-1	-1		
1	1	-1	-1	1	-1	1	-1	1	-1		
-1	1	-1	-1	0	-1	1	1	1	1		

```
#Checking the shape of the dataset
data0.shape

(11055, 32)

#Listing the features of the dataset
data0.columns

Index(['index', 'having_IPhaving_IP_Address', 'URLURL_Length',
       'Shortining_Service', 'having_At_Symbol', 'double_slash_redirecting',
       'Prefix_Suffix', 'having_Sub_Domain', 'SSLfinal_State',
       'Domain_registration_length', 'Favicon', 'port', 'HTTPS_token',
       'Request_URL', 'URL_of_Anchor', 'Links_in_tags', 'SFH',
       'Submitting_to_email', 'Abnormal_URL', 'Redirect', 'on_mouseover',
       'RightClick', 'popUpWidnow', 'Iframe', 'age_of_domain', 'DNSRecord',
       'web_traffic', 'Page_Rank', 'Google_Index', 'Links_pointing_to_page',
       'Statistical_report', 'Result'],
      dtype='object')
```

The above two codes are to check the shape and column of the data frame. By listing the column, it allows access to the features are and needs to be considered for the identification of phished website or not.

The below code lists the information in the dataset in along with the data type of the features enrolled in the column.

```
[ ] #Information about the dataset
data0.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11055 entries, 0 to 11054
Data columns (total 32 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   index                                11055 non-null  int64
1   having_IPhaving_IP_Address           11055 non-null  int64
2   URLURL_Length                        11055 non-null  int64
3   Shortining_Service                   11055 non-null  int64
4   having_At_Symbol                     11055 non-null  int64
5   double_slash_redirecting             11055 non-null  int64
6   Prefix_Suffix                        11055 non-null  int64
7   having_Sub_Domain                    11055 non-null  int64
8   SSLfinal_State                       11055 non-null  int64
9   Domain_registration_length           11055 non-null  int64
10  Favicon                              11055 non-null  int64
11  port                                 11055 non-null  int64
12  HTTPS_token                          11055 non-null  int64
13  Request_URL                          11055 non-null  int64
14  URL_of_Anchor                        11055 non-null  int64
15  Links_in_tags                        11055 non-null  int64
16  SFH                                  11055 non-null  int64
17  Submitting_to_email                  11055 non-null  int64
18  Abnormal_URL                         11055 non-null  int64
19  Redirect                             11055 non-null  int64
20  on_mouseover                         11055 non-null  int64
21  RightClick                           11055 non-null  int64
22  popUpWidnow                          11055 non-null  int64
23  Iframe                              11055 non-null  int64
24  age_of_domain                        11055 non-null  int64
25  DNSRecord                            11055 non-null  int64
26  web_traffic                          11055 non-null  int64
27  Page_Rank                            11055 non-null  int64
28  Google_Index                         11055 non-null  int64
29  Links_pointing_to_page               11055 non-null  int64
30  Statistical_report                   11055 non-null  int64
31  Result                              11055 non-null  int64
dtypes: int64(32)
memory usage: 2.7 MB
```

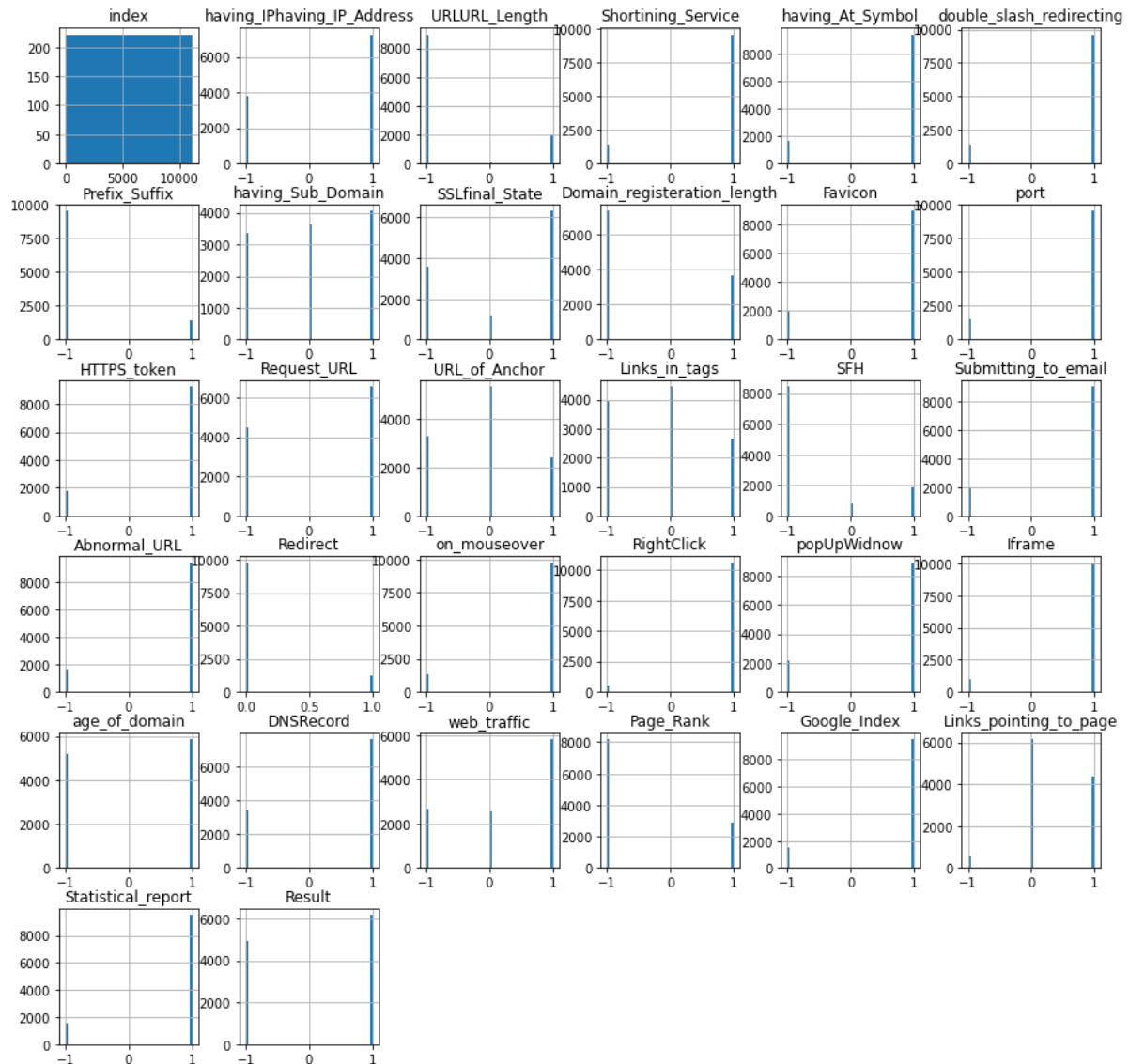
Visualizing the Data

Here, few plots and graphs are displayed to find how the data is distributed and how the features are related to each other.

- Data Distribution Plot



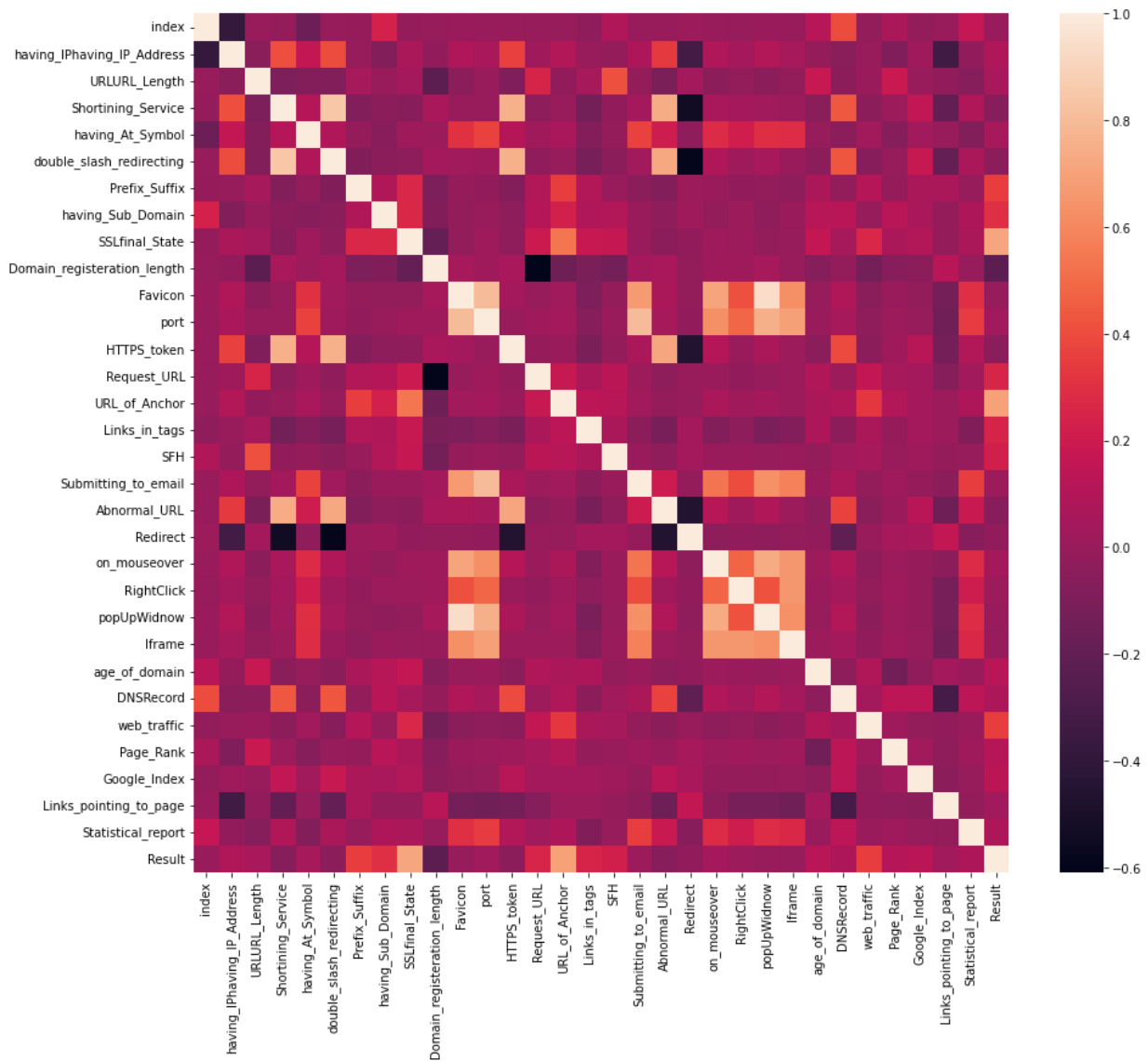
```
#Plotting the data distribution
data0.hist(bins = 50,figsize = (15,15))
plt.show()
```



- Correlation Heatmap

```
#Correlation heatmap

plt.figure(figsize=(15,13))
sns.heatmap(data0.corr())
plt.show()
```



Data Preprocessing and EDA

Here, we clean the data by applying data preprocessing techniques and transform the data to use it in the model.

```
data0.describe()
```

O/P:

	index	having_IPhaving_IP_Address	URLURL_Length	Shortining_Service	having_At_Symbol	double_slash_redirecting	Prefix_Suffix	having_Sub_Domain	SSLfinal_State	Domain_registration_lengt
count	11055.000000	11055.000000	11055.000000	11055.000000	11055.000000	11055.000000	11055.000000	11055.000000	11055.000000	11055.000000
mean	5528.000000	0.313795	-0.633198	0.738761	0.700588	0.741474	-0.734962	0.063953	0.250927	-0.33677
std	3191.447947	0.949534	0.786095	0.673998	0.713598	0.671011	0.678139	0.817518	0.911892	0.94162
min	1.000000	-1.000000	-1.000000	-1.000000	-1.000000	-1.000000	-1.000000	-1.000000	-1.000000	-1.000000
25%	2764.500000	-1.000000	-1.000000	1.000000	1.000000	1.000000	-1.000000	-1.000000	-1.000000	-1.000000
50%	5528.000000	1.000000	-1.000000	1.000000	1.000000	1.000000	-1.000000	0.000000	1.000000	-1.000000
75%	8291.500000	1.000000	-1.000000	1.000000	1.000000	1.000000	-1.000000	1.000000	1.000000	1.000000
max	11055.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

8 rows x 11 columns

popUpWinnow	Iframe	age_of_domain	DNSRecord	web_traffic	Page_Rank	Google_Index	Links_pointing_to_page	Statistical_report	Result
11055.000000	11055.000000	11055.000000	11055.000000	11055.000000	11055.000000	11055.000000	11055.000000	11055.000000	11055.000000
0.613388	0.816915	0.061239	0.377114	0.287291	-0.483673	0.721574	0.344007	0.719584	0.113885
0.789818	0.576784	0.998168	0.926209	0.827733	0.875289	0.692369	0.569944	0.694437	0.993539
-1.000000	-1.000000	-1.000000	-1.000000	-1.000000	-1.000000	-1.000000	-1.000000	-1.000000	-1.000000
1.000000	1.000000	-1.000000	-1.000000	0.000000	-1.000000	1.000000	0.000000	1.000000	-1.000000
1.000000	1.000000	1.000000	1.000000	1.000000	-1.000000	1.000000	0.000000	1.000000	1.000000
1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

The above obtained result shows that most of the data is made up of 0's & 1's except 'Domain' & 'URL_Depth' columns. The domain column doesn't have significance to the machine learning model training. So dropping 'Domain' column from the dataset doesn't affect the functionalities of the code

```
[ ] #Dropping the Domain column
data = data0.drop(['Shortining_Service'], axis = 1).copy()
```

This leaves us with 16 features and a targeted column. The 'URL_Depth' maximum value is 20. According to the understanding of the code, there is no necessity to change the column.

```
#checking the data for null or missing values
data.isnull().sum()

index      0
having_IPhaving_IP_Address  0
URLURL_Length  0
having_At_Symbol  0
double_slash_redirecting  0
Prefix_Suffix  0
having_Sub_Domain  0
SSLfinal_State  0
Domain_registration_length  0
Favicon  0
port  0
HTTPS_token  0
Request_URL  0
URL_of_Anchor  0
Links_in_tags  0
SFH  0
Submitting_to_email  0
Abnormal_URL  0
Redirect  0
on_mouseover  0
RightClick  0
popUpWidnow  0
Iframe  0
age_of_domain  0
DNSRecord  0
web_traffic  0
Page_Rank  0
Google_Index  0
Links_pointing_to_page  0
Statistical_report  0
Result  0
dtype: int64
```

In the feature extraction file, the extracted features of legitimate and phishing URL datasets are just concatenated without any shuffling. This resulted in 500 rows of legitimate URL data and bottom 5000 phishing URL data

To even out the distribution while splitting the data into training and testing sets, we need to shuffle it. This even evades the case of overfitting while model training.

```
# shuffling the rows in the dataset so that when splitting the train and test set are equally distributed
data = data.sample(frac=1).reset_index(drop=True)
data.head()
```

O/P:

index	having_IP	having_IP_Address	URLURL_Length	having_At_Symbol	double_slash_redirecting	Prefix_Suffix	having_Sub_Domain	SSLfinal_State	Domain_registration_length
0	812	1	-1	1	1	-1	1	-1	1
1	10887	-1	-1	1	1	-1	1	-1	-1
2	6056	-1	-1	1	1	-1	0	-1	1
3	7537	-1	1	1	1	-1	0	-1	-1
4	2986	1	-1	1	1	-1	1	1	-1

5 rows x 10 columns

Favicon	...	popUpWidnow	Iframe	age_of_domain	DNSRecord	web_traffic	Page_Rank	Google_Index	Links_pointing_to_page	Statistical_report	Result
1	...	1	1	-1	-1	1	-1	1	0	1	-1
-1	...	-1	1	-1	1	0	-1	1	1	-1	-1
1	...	1	1	-1	1	1	1	1	1	1	-1
1	...	1	1	1	1	-1	-1	1	0	1	-1
1	...	1	1	1	-1	-1	-1	1	1	1	1

From the above execution, it is clear that the data does not have any missing values.

By this, the data is thoroughly preprocessed and is ready for training.

Splitting the data

```
# Splitting the dataset into dependant and independant fetature

X = data.drop(["having_At_Symbol"],axis =1)
y = data["having_At_Symbol"]

[ ] # Splitting the dataset into train and test sets: 80-20 split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,test_size = 0.2, random_state = 12)
X_train.shape, X_test.shape

((8844, 30), (2211, 30))

[ ] x=data.iloc[:,1:31].values
y=data.iloc[:,-1].values

[ ] print(x)

[[ 1 -1  1 ...  0  1 -1]
 [-1 -1  1 ...  1 -1 -1]
 [-1 -1  1 ...  1  1 -1]
 ...
 [ 1 -1  1 ...  1  1 -1]
 [-1 -1  1 ...  1  1  1]
 [ 1 -1 -1 ...  1  1 -1]]

[ ] print(y)

[-1 -1 -1 ... -1  1 -1]
```

```
[ ] X_train.shape, X_test.shape

((8844, 30), (2211, 30))

[ ] y_train.shape,y_test.shape

((8844,), (2211,))

from sklearn.metrics import accuracy_score
from xgboost import XGBClassifier

use_label_encoder=False
xgb = XGBClassifier(learning_rate=0.4,max_depth=7)

xgb.fit(X_train, y_train)

/opt/conda/envs/Python-3.9/lib/python3.9/site-packages/xgboost/sklearn.py:1224: UserWarning:
  warnings.warn(label_encoder_deprecation_msg, UserWarning)
[14:09:31] WARNING: /opt/conda/conda-bld/xgboost-ext_1658814095305/work/src/learner.cc:1115:
XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
               colsample_bynode=1, colsample_bytree=1, enable_categorical=False,
               gamma=0, gpu_id=-1, importance_type=None,
               interaction_constraints='', learning_rate=0.4, max_delta_step=0,
               max_depth=7, min_child_weight=1, missing=nan,
               monotone_constraints=(), n_estimators=100, n_jobs=56,
               num_parallel_tree=1, predictor='auto', random_state=0,
               reg_alpha=0, reg_lambda=1, scale_pos_weight=1, subsample=1,
               tree_method='exact', validate_parameters=1, verbosity=None)
```

```
[ ] y_test1 = xgb.predict(X_test)
    y_train1 = xgb.predict(X_train)

[ ] #computing the accuracy of the model performance

acc_train = accuracy_score(y_train,y_train1)
acc_test = accuracy_score(y_test,y_test1)

print("XGBoost: Accuracy on training Data: {:.3f}".format(acc_train))
print("XGBoost : Accuracy on test Data: {:.3f}".format(acc_test))

XGBoost: Accuracy on training Data: 0.995
XGBoost : Accuracy on test Data: 0.928
```

```
❶ !pip install ibm_watson_machine_learning

Requirement already satisfied: ibm_watson_machine_learning in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (1.0.257)
Requirement already satisfied: packaging in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm_watson_machine_learning) (21.3)
Requirement already satisfied: pandas<1.5.0,>=0.24.2 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm_watson_machine_learning) (1.3.4)
Requirement already satisfied: lmnd in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm_watson_machine_learning) (0.3.2)
Requirement already satisfied: requests in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm_watson_machine_learning) (2.26.0)
Requirement already satisfied: certifi in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm_watson_machine_learning) (2022.9.24)
Requirement already satisfied: urllib3 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm_watson_machine_learning) (1.26.7)
Requirement already satisfied: tabulate in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm_watson_machine_learning) (0.8.9)
Requirement already satisfied: importlib-metadata in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm_watson_machine_learning) (4.8.2)
Requirement already satisfied: ibm-cos-sdk==2.11.* in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm_watson_machine_learning) (2.11.0)
Requirement already satisfied: ibm-cos-sdk-s3transfer==2.11.0 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm-cos-sdk==2.11.*->ibm_watson_machine_learning) (2.11.0)
Requirement already satisfied: ibm-cos-sdk-core==2.11.0 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm-cos-sdk==2.11.*->ibm_watson_machine_learning) (2.11.0)
Requirement already satisfied: jeepney<1.0.0,>=0.7.1 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm-cos-sdk==2.11.*->ibm_watson_machine_learning) (0.10.0)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from ibm-cos-sdk-core==2.11.0->ibm_watson_machine_learning) (2.8.1)
Requirement already satisfied: pytz>=2017.3 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from pandas<1.5.0,>=0.24.2->ibm_watson_machine_learning) (2021.3)
Requirement already satisfied: numpy>=1.17.3 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from pandas<1.5.0,>=0.24.2->ibm_watson_machine_learning) (1.20.3)
Requirement already satisfied: six>=1.5 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from python-dateutil<3.0.0,>=2.1->ibm-cos-sdk-core==2.11.0->ibm_watson_machine_learning) (1.16.0)
Requirement already satisfied: charset-normalizer<2.0.0 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from requests->ibm_watson_machine_learning) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from requests->ibm_watson_machine_learning) (3.3)
Requirement already satisfied: sipp>=0.5 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from importlib-metadata->ibm_watson_machine_learning) (3.6.0)
Requirement already satisfied: pyparsing<3.0.5,>=2.0.2 in /opt/conda/envs/Python-3.9/lib/python3.9/site-packages (from packaging->ibm_watson_machine_learning) (3.0.4)
```

```
[ ] from ibm_watson_machine_learning import APIClient
    wml_credentials={
        "url":"https://us-south.ml.cloud.ibm.com",
        "apikey":"NgTkkHUgkAIRAAMBzOH0mVh6-V9x_TXd_RaaGFNedALP"
    }
    client=APIClient(wml_credentials)

[ ] def guid_from_space_name(client, space_name):
    space = client.spaces.get_details()
    return(next(item for item in space['resources'] if item['entity']['name'] == space_name)['metadata']['id'])

[ ] space_uid = guid_from_space_name(client,'Phishing_Model')
    print("Space UID = " + space_uid)

Space UID = 016ealee-2e6f-4a9c-b90e-b936efc959c7

▶ client.set.default_space(space_uid)

👤 'SUCCESS'
```

+ Code + Text

client.software_specifications.list()

NAME	ASSET_ID	TYPE
default_py3.6	0062b8c9-8b7d-44a0-a9b9-46c416adcbd9	base
kernel-spark3.2-scala2.12	020d69ce-7ac1-5e68-ac1a-31189867356a	base
pytorch-onnx_1.3-py3.7-edt	069ea134-3346-5748-b513-49120e15d288	base
scikit-learn_0.20-py3.6	09c5ald0-9c1e-4473-a344-eb7b665ff687	base
spark-mllib_3.0-scala_2.12	09f4cff0-90a7-5899-b9ed-1ef348aebdee	base
pytorch-onnx_rt22.1-py3.9	0b848dd4-e681-5599-be41-b5f6fccc6471	base
ai-function_0.1-py3.6	0cdb0f1e-5376-4f4d-92dd-da3b69aa9bda	base
shiny-r3.6	0e6e79df-875e-4f24-8ae9-62dcc2148306	base
tensorflow_2.4-py3.7-horovod	1092590a-307d-563d-9b62-4eb7d64b3f22	base
pytorch_1.1-py3.6	10ac12d6-6b30-4ccd-8392-3e922c096a92	base
tensorflow_1.15-py3.6-ddl	111e41b3-de2d-5422-a4d6-bf776828c4b7	base
autoai-kb_rt22.2-py3.10	125b6d9a-5b1f-5e8d-972a-b251688ccf40	base
runtime-22.1-py3.9	12b83a17-24d8-5082-900f-0ab31fbfd3cb	base
scikit-learn_0.22-py3.6	154010fa-5b3b-4ac1-82af-4d5ee5abbc85	base
default_r3.6	1b70aec3-ab34-4b87-8aa0-a4a3c8296a36	base
pytorch-onnx_1.3-py3.6	1bc6029a-cc97-56da-b8e0-39c3880dbbe7	base
kernel-spark3.3-r3.6	1c9e5454-f216-59dd-a20e-474a5cdf5988	base
pytorch-onnx_rt22.1-py3.9-edt	1d362186-7ad5-5b59-8b6c-9d0880bde37f	base
tensorflow_2.1-py3.6	1eb25b84-d6ed-5dde-b6a5-3fbdf1665666	base
spark-mllib_3.2	20047f72-0a98-58c7-9ff5-a77b012eb8f5	base
tensorflow_2.4-py3.8-horovod	217c16f6-178f-56bf-824a-b19f20564c49	base
runtime-22.1-py3.9-cuda	26215f05-08c3-5a41-a1b0-da66306ce658	base
do_py3.8	295addb5-9ef9-547e-9bf4-92ae3563e720	base
autoai-ts_3.8-py3.8	2aa0c932-798f-5ae9-abd6-15e0c2402fb5	base
tensorflow_1.15-py3.6	2b73a275-7cbf-420b-a912-eae7f436e0bc	base
kernel-spark3.3-py3.9	2b7961e2-e3b1-5a8c-a491-482c8368839a	base
pytorch_1.2-py3.6	2c8ef57d-2687-4b7d-acce-01f94976dac1	base
spark-mllib_2.3	2e51f700-bca0-4b0d-88dc-5c6791338875	base
pytorch-onnx_1.1-py3.6-edt	32983cea-3f32-4400-8965-dde874a8d67e	base
spark-mllib_3.0-py37	36507ebe-8770-55ba-ab2a-eafe787600e9	base
spark-mllib_2.4	390d21f8-e58b-4fac-9c55-d7ceda621326	base
autoai-ts_rt22.2-py3.10	396b2e83-0953-5b86-9a55-7ce1628a406f	base
xgboost_0.82-py3.6	39e31acd-5f30-41dc-ae44-60233c80306e	base
pytorch-onnx_1.2-py3.6-edt	40589d0e-7019-4e28-8daa-fb03b6f4fe12	base
pytorch-onnx_rt22.2-py3.10	40e73f55-783a-5535-b3fa-0c8b94291431	base

pytorch-onnx_rt22.2-py3.10	40e73f55-783a-5535-b3fa-0c8b94291431	base
default_r36py38	41c247d3-45f8-5a71-b065-8580229facf0	base
autoai-ts_rt22.1-py3.9	4269d26e-07ba-5d40-8f66-2d495b0c71f7	base
autoai-obm_3.0	42b92e18-d9ab-567f-988a-4240baled5f7	base
pmml-3.0_4.3	493bcb95-16f1-5bc5-bee8-81b8af80e9c7	base
spark-mllib_2.4-r_3.6	49403dff-92e9-4c87-a3d7-a42d0021c095	base
xgboost_0.90-py3.6	4ff8d6c2-1343-4c18-85e1-689c965304d3	base
pytorch-onnx_1.1-py3.6	50f95b2a-bc16-43bb-bc94-b0bed208c60b	base
autoai-ts_3.9-py3.8	52c57136-80fa-572e-8728-a5e7cbb42cde	base
spark-mllib_2.4-scala_2.11	55a70f99-7320-4be5-9fb9-9edb5a443af5	base
spark-mllib_3.0	5c1b0ca2-4977-5c2e-9439-ffd44ea8ffe9	base
autoai-obm_2.0	5c2e37fa-80b8-5e77-840f-d912469614ee	base
spss-modeler_18.1	5c3cad7e-507f-4b2a-a9a3-ab53a21dee8b	base
cuda-py3.8	5d3232bf-c86b-5df4-a2cd-7bb870a1cd4e	base
runtime-22.2-py3.10-xc	5e8cddff-db4a-5a6a-b8aa-2d4af9864dab	base
autoai-kb_3.1-py3.7	632d4b22-10aa-5180-88f0-f52dfb6444d7	base

Note: Only first 50 records were displayed. To display more use 'limit' parameter.

```
[ ] software_spec_uid = client.software_specifications.get_uid_by_name("runtime-22.1-py3.9")
software_spec_uid

'12b83a17-24d8-5082-900f-0ab31fbfd3cb'

[ ] model_details = client.repository.store_model(model=xgb,meta_props={
client.repository.ModelMetaNames.NAME:"Model_Building",
client.repository.ModelMetaNames.TYPE:"scikit-learn_1.0",
client.repository.ModelMetaNames.SOFTWARE_SPEC_UID:software_spec_uid }
)

model_id =client.repository.get_model_uid(model_details)

This method is deprecated, please use get_model_id()

model_id

'55d78968-4c45-45dd-a6ae-cd34e8e2eb16'
```

```
ls -l
```

Machine Learning Models and Training

From the data set above, it is clear that this is a supervised machine learning task. There are two major types of supervised machine learning problems, called

- Classification and
- Regression.

This data comes under classification problem, as the input URL is classified Phishing (1) or Legitimate (0). The supervised machine learning models (classification) considered to train the dataset in this notebook are:

- Decision Tree
- Random Forest
- XGBoost

```
[ ] #importing packages
    from sklearn.metrics import accuracy_score

▶ # Creating holders to store the model performance results
ML_Model = []
acc_train = []
acc_test = []

#function to call for storing the results
def storeResults(model, a,b):
    ML_Model.append(model)
    acc_train.append(round(a, 3))
    acc_test.append(round(b, 3))
```

These three are the Lo-key considered algorithms for this particular type testing which provides better accuracy and F_number results

1. Decision Tree Model:

7.1. Decision Tree Classifier

Decision trees are widely used models for classification and regression tasks. Essentially, they learn a hierarchy of if/else questions, leading to a decision. Learning a decision tree means learning the sequence of if/else questions that gets us to the true answer most quickly.

In the machine learning setting, these questions are called tests (not to be confused with the test set, which is the data we use to test to see how generalizable our model is). To build a tree, the algorithm searches over all possible tests and finds the one that is most informative about the target variable.

```
▶ # Decision Tree model
from sklearn.tree import DecisionTreeClassifier

# instantiate the model
tree = DecisionTreeClassifier(max_depth = 5)
# fit the model
tree.fit(X_train, y_train)
```

```
DecisionTreeClassifier(max_depth=5)
```

```
[ ] #predicting the target value from the model for the samples
y_test_tree = tree.predict(X_test)
y_train_tree = tree.predict(X_train)
```

Performance Evaluation:

```
[ ] #computing the accuracy of the model performance
acc_train_tree = accuracy_score(y_train,y_train_tree)
acc_test_tree = accuracy_score(y_test,y_test_tree)

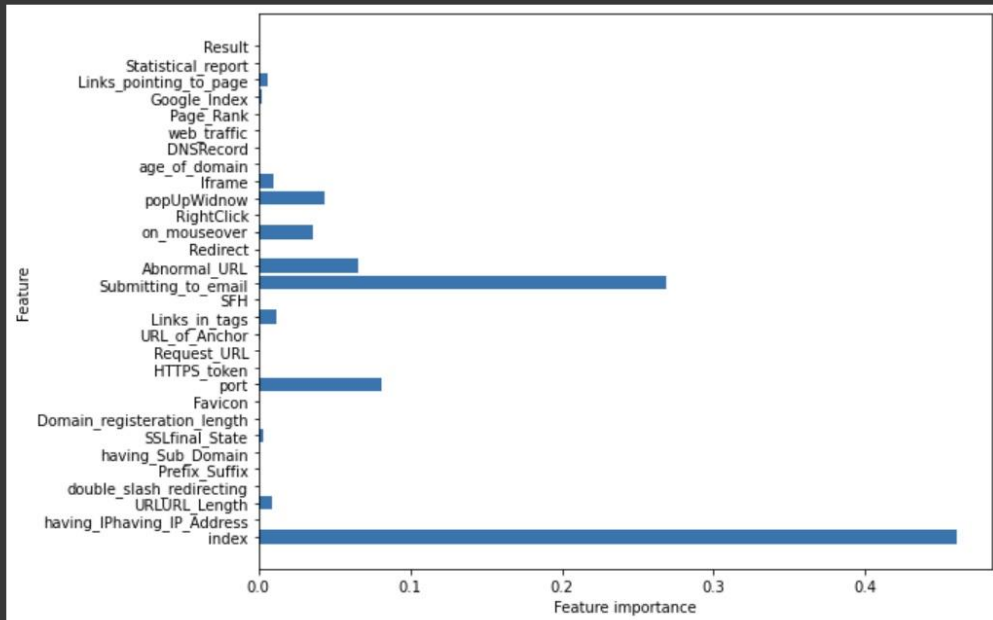
print("Decision Tree: Accuracy on training Data: {:.3f}".format(acc_train_tree))
print("Decision Tree: Accuracy on test Data: {:.3f}".format(acc_test_tree))
```

```
Decision Tree: Accuracy on training Data: 0.928
Decision Tree: Accuracy on test Data: 0.918
```

```

▶ #checking the feature improtance in the model
plt.figure(figsize=(9,7))
n_features = X_train.shape[1]
plt.barh(range(n_features), tree.feature_importances_, align='center')
plt.yticks(np.arange(n_features), X_train.columns)
plt.xlabel("Feature importance")
plt.ylabel("Feature")
plt.show()

```



Storing the results:

```

[ ] #storing the results. The below mentioned order of parameter passing is important.
    #Caution: Execute only once to avoid duplications.
    storeResults('Decision Tree', acc_train_tree, acc_test_tree)

```

2. Random Forest Classifier:

7.2. Random Forest Classifier

Random forests for regression and classification are currently among the most widely used machine learning methods. A random forest is essentially a collection of decision trees, where each tree is slightly different from the others. The idea behind random forests is that each tree might do a relatively good job of predicting, but will likely overfit on part of the data.

If we build many trees, all of which work well and overfit in different ways, we can reduce the amount of overfitting by averaging their results. To build a random forest model, you need to decide on the number of trees to build (the `n_estimators` parameter of `RandomForestRegressor` or `RandomForestClassifier`). They are very powerful, often work well without heavy tuning of the parameters, and don't require scaling of the data.

```
[ ] # Random Forest model
    from sklearn.ensemble import RandomForestClassifier

    # instantiate the model
    forest = RandomForestClassifier(max_depth=5)

    # fit the model
    forest.fit(X_train, y_train)

    RandomForestClassifier(max_depth=5)

[ ] #predicting the target value from the model for the samples
    y_test_forest = forest.predict(X_test)
    y_train_forest = forest.predict(X_train)
```

Performance Evaluation:

```
[ ] #computing the accuracy of the model performance
    acc_train_forest = accuracy_score(y_train, y_train_forest)
    acc_test_forest = accuracy_score(y_test, y_test_forest)

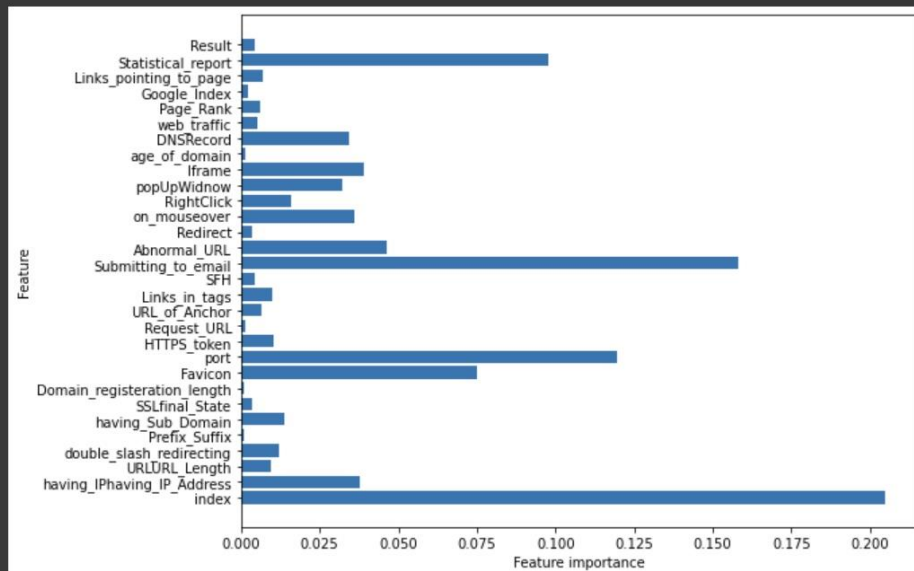
    print("Random forest: Accuracy on training Data: {:.3f}".format(acc_train_forest))
    print("Random forest: Accuracy on test Data: {:.3f}".format(acc_test_forest))

    Random forest: Accuracy on training Data: 0.913
    Random forest: Accuracy on test Data: 0.905
```

```

▶ #checking the feature improtance in the model
plt.figure(figsize=(9,7))
n_features = X_train.shape[1]
plt.barh(range(n_features), forest.feature_importances_, align='center')
plt.yticks(np.arange(n_features), X_train.columns)
plt.xlabel("Feature importance")
plt.ylabel("Feature")
plt.show()

```



Storing the results:

```

[ ] #storing the results. The below mentioned order of parameter passing is important.
    #Caution: Execute only once to avoid duplications.
    storeResults('Random Forest', acc_train_forest, acc_test_forest)

```

3. XGBOOST:

7.4. XGBoost Classifier

XGBoost is one of the most popular machine learning algorithms these days. XGBoost stands for eXtreme Gradient Boosting. Regardless of the type of prediction task at hand; regression or classification. XGBoost is an implementation of gradient boosted decision trees designed for speed and performance.

```
#XGBoost Classification model
from xgboost import XGBClassifier
use_label_encoder=False
# instantiate the model
xgb = XGBClassifier(learning_rate=0.4,max_depth=7)
#fit the model
xgb.fit(X_train, y_train)

[15:15:15] WARNING: /opt/conda/conda-bld/xgboost-ext_1658814095305/work/src/learner.cc:1115: Starting in XGBoost 1.3.0,

[ ] #predicting the target value from the model for the samples
y_test_xgb = xgb.predict(X_test)
y_train_xgb = xgb.predict(X_train)
```

Performance Evaluation:

```
[ ] #computing the accuracy of the model performance
acc_train_xgb = accuracy_score(y_train,y_train_xgb)
acc_test_xgb = accuracy_score(y_test,y_test_xgb)

print("XGBoost: Accuracy on training Data: {:.3f}".format(acc_train_xgb))
print("XGBoost : Accuracy on test Data: {:.3f}".format(acc_test_xgb))
```

Storing the results:

click to expand

```
[ ] #storing the results. The below mentioned order of parameter passing is important.
#Caution: Execute only once to avoid duplications.
storeResults('XGBoost', acc_train_xgb, acc_test_xgb)
```

Comparison of Models

To compare the models performances, a data frame is created. The column of this data frame are the lists created to store the results of the model.

```
[ ] #creating dataframe
    results = pd.DataFrame({ 'ML Model': ML_Model,
                             'Train Accuracy': acc_train,
                             'Test Accuracy': acc_test})
    results

[ ] #Sorting the dataframe on accuracy
    results.sort_values(by=['Test Accuracy', 'Train Accuracy'], ascending=False)
```

For the above comparison, it is clear that the XGBoost Classifier works well with this dataset.
So, saving the model for future use.

```
[ ] # save XGBoost model to file
    import pickle
    pickle.dump(xgb, open("XGBoostClassifier.pickle.dat", "wb"))

[ ] # load model from file
    loaded_model = pickle.load(open("XGBoostClassifier.pickle.dat", "rb"))
    loaded_model
```

Double-click (or enter) to edit

1. Finally, the Ultimate goal of this project is to explore various methods of algorithms in machine learning models and the perform Data Analysis on phished website dataset and understanding their features.
2. Jupyter notebook allowed to access all the contents with test and run the various algorithms for classification.
3. Important characteristic features like "HTTPS", "AnchorURL", "WebsiteTraffic" have desired variations in finding a website is phished or not.
4. By choosing the best model helps in predicting whether a site URL is phishing or not accurately.

7.2 Feature 2:

CODE:

```
# importing required packages for this section

from urllib.parse import urlparse,urlencode

import ipaddress

import re
```

```
"""#### **3.1.1. Domain of the URL**
```

Here, we are just extracting the domain present in the URL. This feature doesn't have much significance in the training. May even be dropped while training the model.

```
"""
```

```
'''
```

```
# 1.Domain of the URL (Domain)
```

```
def getDomain(url):
```

```
    domain = urlparse(url).netloc
```

```
    if re.match(r"^\www.",domain):
```

```
        domain = domain.replace("www.", "")
```

```
    return domain'''
```

```
"""#### **3.1.2. IP Address in the URL**
```

Checks for the presence of IP address in the URL. URLs may have IP address instead of domain name. If an IP address is used as an alternative of the domain name in the URL, we can be sure that someone is trying to steal personal information with this URL.

If the domain part of URL has IP address, the value assigned to this feature is 1 (phishing) or else 0 (legitimate).

```
"""
```

```
# 2.Checks for IP address in URL (Have_IP)
```

```
def havingIP(url):
```

```
    try:
```

```
        ipaddress.ip_address(url)
```

```
        ip = 1
```

```
    except:
```

```
        ip = 0
```

```
    return ip
```

```
##### **3.1.3. "@" Symbol in URL**
```

Checks for the presence of '@' symbol in the URL. Using "@" symbol in the URL leads the browser to ignore everything preceding the "@" symbol and the real address often follows the "@" symbol.

If the URL has '@' symbol, the value assigned to this feature is 1 (phishing) or else 0 (legitimate).

```
"""
```

```
# 3.Checks the presence of @ in URL (Have_At)
```

```
def haveAtSign(url):
```

```
    if "@" in url:
```

```
        at = 1
```

```
    else:
```

```
at = 0
```

```
return at
```

```
##### **3.1.4. Length of URL**
```

Computes the length of the URL. Phishers can use long URL to hide the doubtful part in the address bar. In this project, if the length of the URL is greater than or equal 54 characters then the URL classified as phishing otherwise legitimate.

If the length of URL ≥ 54 , the value assigned to this feature is 1 (phishing) or else 0 (legitimate).

```
#####
```

```
# 4.Finding the length of URL and categorising (URL_Length)
```

```
def getLength(url):
```

```
    if len(url) < 54:
```

```
        length = 0
```

```
    else:
```

```
        length = 1
```

```
    return length
```

```
##### **3.1.5. Depth of URL**
```

Computes the depth of the URL. This feature calculates the number of sub pages in the given url based on the '/'.
The value of feature is a numerical based on the URL.

```
#####
```

```
# 5.Gives number of '/' in URL (URL_Depth)
```

```
def getDepth(url):
```

```
    s = urlparse(url).path.split('/')

```

```
    depth = 0
```

```
    for j in range(len(s)):
```

```
        if len(s[j]) != 0:
```

```
            depth = depth+1
```

```
    return depth
```

```
##### **3.1.6. Redirection "/" in URL**
```

Checks the presence of "/" in the URL. The existence of "/" within the URL path means that the user will be redirected to another website. The location of the "/" in URL is computed. We find that if the URL starts with "HTTP", that means the "/" should appear in the sixth position. However, if the URL employs "HTTPS" then the "/" should appear in seventh position.

If the "/" is anywhere in the URL apart from after the protocol, the value assigned to this feature is 1 (phishing) or else 0 (legitimate).

```
#####
```

```
# 6.Checking for redirection '/' in the url (Redirection)
```

```
def redirection(url):
```

```
    pos = url.rfind('/')

```

```
    if pos > 6:
```

```
        if pos > 7:
```

```
            return 1
```

```
else:
```

```
    return 0
```

```
else:
```

```
    return 0
```

```
##### **3.1.7. "http/https" in Domain name**
```

Checks for the presence of "http/https" in the domain part of the URL. The Phishers may add the "HTTPS" token to the domain part of a URL in order to trick users.

If the URL has "http/https" in the domain part, the value assigned to this feature is 1 (phishing) or else 0 (legitimate).

```
#####
```

```
# 7.Existence of "HTTPS" Token in the Domain Part of the URL (https_Domain)
```

```
def httpDomain(url):
```

```
    domain = urlparse(url).netloc
```

```
    if 'https' in domain:
```

```
        return 1
```

```
    else:
```

```
        return 0
```

```
##### **3.1.8. Using URL Shortening Services "TinyURL"**
```

URL shortening is a method on the "World Wide Web" in which a URL may be made considerably smaller in length and still lead to the required webpage. This is accomplished by means of an "HTTP Redirect" on a domain name that is short, which links to the webpage that has a long URL.

If the URL is using Shortening Services, the value assigned to this feature is 1 (phishing) or else 0 (legitimate).

```
"""
```

```
#listing shortening services
```

```
shortening_services = r"bit\.ly|goo\.gl|shorte\.st|go2l\.ink|x\.co|ow\.ly|t\.co|tinyurl|tr\.im|is\.gd|cli\.gs|" \
```

```
    r"yfrog\.com|migre\.me|ff\.im|tiny\.cc|url4\.eu|twit\.ac|su\.pr|twurl\.nl|snipurl\.com|" \
```

```
    r"short\.to|BudURL\.com|ping\.fm|post\.ly|Just\.as|bkite\.com|snipr\.com|fic\.kr|loopt\.us|" \
```

```
    r"doiop\.com|short\.ie|kl\.am|wp\.me|rubyurl\.com|om\.ly|to\.ly|bit\.do|t\.co|lnkd\.in|db\.tt|" \
```

```
    r"qr\.ae|adf\.ly|goo\.gl|bitly\.com|cur\.lv|tinyurl\.com|ow\.ly|bit\.ly|ity\.im|q\.gs|is\.gd|" \
```

```
    r"po\.st|bc\.vc|twitthis\.com|u\.to|j\.mp|buzurl\.com|cutt\.us|u\.bb|yourls\.org|x\.co|" \
```

```
    r"prettylinkpro\.com|scrnch\.me|filoops\.info|vzturl\.com|qr\.net|1url\.com|tweez\.me|v\.gd|" \
```

```
    r"tr\.im|link\.zip\.net"
```

```
# 8. Checking for Shortening Services in URL (Tiny_URL)
```

```
def tinyURL(url):
```

```
    match=re.search(shortening_services,url)
```

```
    if match:
```

```
        return 1
```

```
    else:
```

```
        return 0
```

```
##### **3.1.9. Prefix or Suffix "-" in Domain**
```

Checking the presence of '-' in the domain part of URL. The dash symbol is rarely used in legitimate URLs. Phishers tend to add prefixes or suffixes separated by (-) to the domain name so that users feel that they are dealing with a legitimate webpage.

If the URL has '-' symbol in the domain part of the URL, the value assigned to this feature is 1 (phishing) or else 0 (legitimate).

```
"""
```

```
# 9.Checking for Prefix or Suffix Separated by (-) in the Domain (Prefix/Suffix)
```

```
def prefixSuffix(url):
```

```
    if '-' in urlparse(url).netloc:
```

```
        return 1      # phishing
```

```
    else:
```

```
        return 0      # legitimate
```

```
"""### **3.2. Domain Based Features:**
```

Many features can be extracted that come under this category. Out of them, below mentioned were considered for this project.

- * DNS Record
- * Website Traffic
- * Age of Domain
- * End Period of Domain

Each of these features are explained and the coded below:

```
"""
```

```
#!/pip install python-whois
```

```
# importing required packages for this section
```

```
import re
```

```
from bs4 import BeautifulSoup
```

```
#import whois
```

```
import urllib
```

```
import urllib.request
```

```
from datetime import datetime
```

```
##### **3.2.1. DNS Record**
```

For phishing websites, either the claimed identity is not recognised by the WHOIS database or no records founded for the hostname.

If the DNS record is empty or not found then, the value assigned to this feature is 1 (phishing) or else 0 (legitimate).

```
"""
```

```
# 11.DNS Record availability (DNS_Record)
```

```
# obtained in the featureExtraction function itself
```

```
##### **3.2.2. Web Traffic**
```

This feature measures the popularity of the website by determining the number of visitors and the number of pages they visit. However, since phishing websites live for a short period of time, they may

not be recognised by the Alexa database (Alexa the Web Information Company., 1996). By reviewing our dataset, we find that in worst scenarios, legitimate websites ranked among the top 100,000. Furthermore, if the domain has no traffic or is not recognised by the Alexa database, it is classified as “Phishing”.

If the rank of the domain < 100000, the value of this feature is 1 (phishing) else 0 (legitimate).

```
"""
```

```
# 12.Web traffic (Web_Traffic)
```

```
def web_traffic(url):
```

```
    try:
```

```
        #Filling the whitespaces in the URL if any
```

```
        url = urllib.parse.quote(url)
```

```
        rank = BeautifulSoup(urllib.request.urlopen("http://data.alexa.com/data?cli=10&dat=s&url=" +  
url).read(), "xml").find(
```

```
            "REACH")['RANK']
```

```
        rank = int(rank)
```

```
    except TypeError:
```

```
        return 1
```

```
    if rank <100000:
```

```
        return 1
```

```
    else:
```

```
        return 0
```

```
##### **3.2.3. Age of Domain**
```

This feature can be extracted from WHOIS database. Most phishing websites live for a short period of time. The minimum age of the legitimate domain is considered to be 12 months for this project. Age here is nothing but different between creation and expiration time.

If age of domain > 12 months, the value of this feature is 1 (phishing) else 0 (legitimate).

''''''

13.Survival time of domain: The difference between termination time and creation time
(Domain_Age)

```
def domainAge(domain_name):

    creation_date = domain_name.creation_date

    expiration_date = domain_name.expiration_date

    if (isinstance(creation_date,str) or isinstance(expiration_date,str)):

        try:

            creation_date = datetime.strptime(creation_date,'%Y-%m-%d')

            expiration_date = datetime.strptime(expiration_date,"%Y-%m-%d")

        except:

            return 1

    if ((expiration_date is None) or (creation_date is None)):

        return 1

    elif ((type(expiration_date) is list) or (type(creation_date) is list)):

        return 1

    else:

        ageofdomain = abs((expiration_date - creation_date).days)

        if ((ageofdomain/30) < 6):

            age = 1

        else:
```

```

    age = 0

    return age

```

```

##### **3.2.4. End Period of Domain**

```

This feature can be extracted from WHOIS database. For this feature, the remaining domain time is calculated by finding the different between expiration time & current time. The end period considered for the legitimate domain is 6 months or less for this project.

If end period of domain > 6 months, the value of this feature is 1 (phishing) else 0 (legitimate).

```

#####

```

14.End time of domain: The difference between termination time and current time (Domain_End)

```

def domainEnd(domain_name):

```

```

    expiration_date = domain_name.expiration_date

```

```

    if isinstance(expiration_date,str):

```

```

        try:

```

```

            expiration_date = datetime.strptime(expiration_date,"%Y-%m-%d")

```

```

        except:

```

```

            return 1

```

```

    if (expiration_date is None):

```

```

        return 1

```

```

    elif (type(expiration_date) is list):

```

```

        return 1

```

```

    else:

```

```

        today = datetime.now()

```

```

end = abs((expiration_date - today).days)

if ((end/30) < 6):

    end = 0

else:

    end = 1

return end

```

"""## **3.3. HTML and JavaScript based Features**

Many features can be extracted that come under this category. Out of them, below mentioned were considered for this project.

- * IFrame Redirection
- * Status Bar Customization
- * Disabling Right Click
- * Website Forwarding

Each of these features are explained and the coded below:

"""

```

# importing required packages for this section

import requests

```

"""### **3.3.1. IFrame Redirection**

IFrame is an HTML tag used to display an additional webpage into one that is currently shown. Phishers can make use of the “iframe” tag and make it invisible i.e. without frame borders. In this regard, Phishers make use of the “frameBorder” attribute which causes the browser to render a visual delineation.

If the iframe is empty or response is not found then, the value assigned to this feature is 1 (phishing) or else 0 (legitimate).

"""

15. IFrame Redirection (iFrame)

```
def iframe(response):
```

```
    if response == '':
```

```
        return 1
```

```
    else:
```

```
        if re.findall(r"<iframe>|<frameBorder>]", response.text):
```

```
            return 0
```

```
        else:
```

```
            return 1
```

3.3.2. Status Bar Customization

Phishers may use JavaScript to show a fake URL in the status bar to users. To extract this feature, we must dig-out the webpage source code, particularly the “onMouseOver” event, and check if it makes any changes on the status bar

If the response is empty or onmouseover is found then, the value assigned to this feature is 1 (phishing) or else 0 (legitimate).

"""

16.Checks the effect of mouse over on status bar (Mouse_Over)

```
def mouseOver(response):
```

```
    if response == "" :
```

```
        return 1
```

```
    else:
```

```
        if re.findall("<script>.+onmouseover.+</script>", response.text):
```

```
            return 1
```

```
        else:
```

```
            return 0
```

3.3.3. Disabling Right Click

Phishers use JavaScript to disable the right-click function, so that users cannot view and save the webpage source code. This feature is treated exactly as “Using onMouseOver to hide the Link”. Nonetheless, for this feature, we will search for event “event.button==2” in the webpage source code and check if the right click is disabled.

If the response is empty or onmouseover is not found then, the value assigned to this feature is 1 (phishing) or else 0 (legitimate).

#####

17.Checks the status of the right click attribute (Right_Click)

```
def rightClick(response):
```

```
    if response == "":
```

```
        return 1
```

```
    else:
```

```

if re.findall(r"event.button ?== ?2", response.text):

    return 0

else:

    return 1

```

"""### **3.3.4. Website Forwarding**

The fine line that distinguishes phishing websites from legitimate ones is how many times a website has been redirected. In our dataset, we find that legitimate websites have been redirected one time max. On the other hand, phishing websites containing this feature have been redirected at least 4 times.

"""

18.Checks the number of forwarding (Web_Forwards)

```

def forwarding(response):

    if response == "":

        return 1

    else:

        if len(response.history) <= 2:

            return 0

        else:

            return 1

```

"""## **4. Computing URL Features**

Create a list and a function that calls the other functions and stores all the features of the URL in the list. We will extract the features of each URL and append to this list.

"""

```
#Function to extract features
```

```
def featureExtraction(url):
```

```
    features = []
```

```
    #Address bar based features (10)
```

```
    #features.append(getDomain(url))
```

```
    features.append(havingIP(url))
```

```
    features.append(haveAtSign(url))
```

```
    features.append(getLength(url))
```

```
    features.append(getDepth(url))
```

```
    features.append(redirection(url))
```

```
    features.append(httpDomain(url))
```

```
    features.append(tinyURL(url))
```

```
    features.append(prefixSuffix(url))
```

```
    #Domain based features (4)
```

```
    dns = 0
```

```
    try:
```

```
        domain_name = whois.whois(urlparse(url).netloc)
```

```
    except:
```

```
        dns = 1
```

```
    features.append(dns)
```

```
    features.append(web_traffic(url))
```

```
    features.append(1 if dns == 1 else domainAge(domain_name))
```

```

features.append(1 if dns == 1 else domainEnd(domain_name))

# HTML & Javascript based features

try:

    response = requests.get(url)

except:

    response = ""

features.append(iframe(response))

features.append(mouseOver(response))

features.append(rightClick(response))

features.append(forwarding(response))

return features

#converting the list to data frame

feature_names = ['Domain', 'Have_IP', 'Have_At', 'URL_Length', 'URL_Depth','Redirection',

                 'https_Domain', 'TinyURL', 'Prefix/Suffix', 'DNS_Record', 'Web_Traffic',

                 'Domain_Age', 'Domain_End', 'iFrame', 'Mouse_Over','Right_Click', 'Web_Forwards',

'Label']

```

8. TESTING

8.1 Test Cases:

Test case ID	Feature Type	Component	Test Scenario	Pre-Requlite	Steps To Execute	Test Data	Expected Result	Actual Result	Status	Comments	TC for Automation (Y/N)	BUG ID	Executed By
Login_Page_TC_QQ 1	Functional	Home Page	After user typing the URL in the box its verified is the user is directed to land page		1. open the website by clicking the URL 2. Type or copy paste the test URL 3. Verify whether it is processing or not.	http:// 127.0.0.1:5000/	Webpage should be viewed in display	Acquired expected result	Pass		N		Raghul C
Login_Page_TC_QQ 2	User interface	Home Page	User interface element is verified		1. open the website by clicking the URL 2. Type or copy paste the test URL 3. Check whether the button is responded or not. 4. Reload and Test Simultaneously	http:// 127.0.0.1:5000/	Get the acknowledgement	Acquired expected result	Pass		N		Raya Rohit Yadav
Login_Page_TC_QQ 3	Functional	Home page	Checking the test link is legitimate or not		1. open the website by clicking the URL 2. Type or copy paste the test URL 3. Check the website is legitimate or not with help of colour change and message displayed 4. Observe the results	http:// 127.0.0.1:5000/	user should be informed is the URL is legitimate or not	Acquired expected result	Pass		N		Prasanna Srinivas. R
Login_Page_TC_QQ 4	Functional	Home Page	Checking is user able to access the legitimate site or not		1. open the website by clicking the URL 2. Type or copy paste the test URL 3. Check the website is legitimate or not with help of colour change and message displayed 4. Continue if the website is legitimate or be cautious if it is not legitimate.	http:// 127.0.0.1:5000/	Application should show that Safe Webpage or Unsafe.	Acquired expected result	Pass		N		Sakthivel. P
Login_Page_TC_QQ 5	Functional	Home Page	Many other URL links is been tested		1. open the website by clicking the URL 2. Type or copy paste the test URL 3. Check the website is legitimate or not. 4. Continue if the website is secure or be cautious if it is not secure	http:// 127.0.0.1:5000/	User should be notified is the site legitimate or not with change in colour(Red/Green) and a message	Acquired expected result	Pass		N		Raya Rohit Yadav

8.2 User Acceptance testing:

1. Purpose of Document

The purpose of this document is to briefly explain the test coverage and open issues of the [Web Phishing Detection] project at the time of the release to User Acceptance Testing (UAT).

2. Defect Analysis

This report shows the number of resolved or closed bugs at each severity level, and how they were resolved

Resolution	Severity1	Severity 2	Severity 3	Severity 4	Subtotal
By Design	10	2	4	20	20
Duplicate	1	3	0	0	4
External	3	1	0	2	6
Fixed	8	4	8	16	36
Not Reproduced	0	0	2	1	3
Skipped	0	0	1	0	1
Wont Fix	0	0	0	2	2
Totals	22	10	15	40	72

3. Test Case Analysis

Section	Total Cases	Not Tested	Fail	Pass
Print Engine	10	0	0	9
Client Application	40	0	0	40
Security	6	0	0	5
Outsource Shipping	4	0	0	4
Exception Reporting	10	0	0	9
Final Report Output	10	0	0	10
Version Control	3	0	0	3

9. RESULTS

9.1 Performance Metrics:

S.No.	Parameter	Values	Screenshot
1.	Metrics	Classification Model: XGB(Gradient Boost classification) Accuracy Score-99.5%	Performance Evaluation: <pre>#computing the accuracy of the model performance acc_train_xgb = accuracy_score(y_train,y_train_xgb) acc_test_xgb = accuracy_score(y_test,y_test_xgb) print("XGBoost: Accuracy on training Data: {:.3f}".format(acc_train_xgb)) print("XGBoost : Accuracy on test Data: {:.3f}".format(acc_test_xgb)) XGBoost: Accuracy on training Data: 0.995 XGBoost : Accuracy on test Data: 0.937</pre>
2.	Tune the Model	Hyper Parameter Tuning – 99% Validation Method - KFOLD & Cross Validation Method	Wilcoxon signed-rank test <pre>In [78]: #KFOLD and Cross Validation Model from scipy.stats import wilcoxon from sklearn.datasets import load_iris from sklearn.ensemble import GradientBoostingClassifier from xgboost import XGBClassifier from sklearn.model_selection import cross_val_score, KFold # Load the dataset X = load_iris().data y = load_iris().target # Prepare models and select your CV method model1 = GradientBoostingClassifier(n_estimators=100) model2 = XGBClassifier(n_estimators=100) kf = KFold(n_splits=20, random_state=None) # Extract results for each model on the same folds results_model1 = cross_val_score(model1, X, y, cv=kf) results_model2 = cross_val_score(model2, X, y, cv=kf) stat, p = wilcoxon(results_model1, results_model2, zero_method='split') stat Out[78]: 95.8</pre>

1. METRICS:

CLASSIFICATION REPORT:

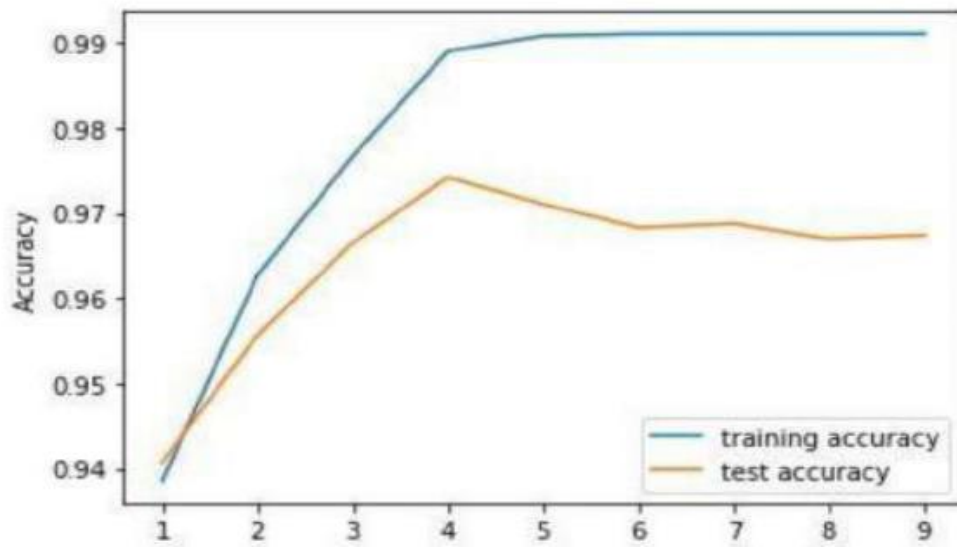
Performance Evaluation:

```
#computing the accuracy of the model performance
acc_train_xgb = accuracy_score(y_train,y_train_xgb)
acc_test_xgb = accuracy_score(y_test,y_test_xgb)

print("XGBoost: Accuracy on training Data: {:.3f}".format(acc_train_xgb))
print("XGBoost : Accuracy on test Data: {:.3f}".format(acc_test_xgb))
```

XGBoost: Accuracy on training Data: 0.995
 XGBoost : Accuracy on test Data: 0.937

2. PERFORMANCE & FEATURE ACCURACY:



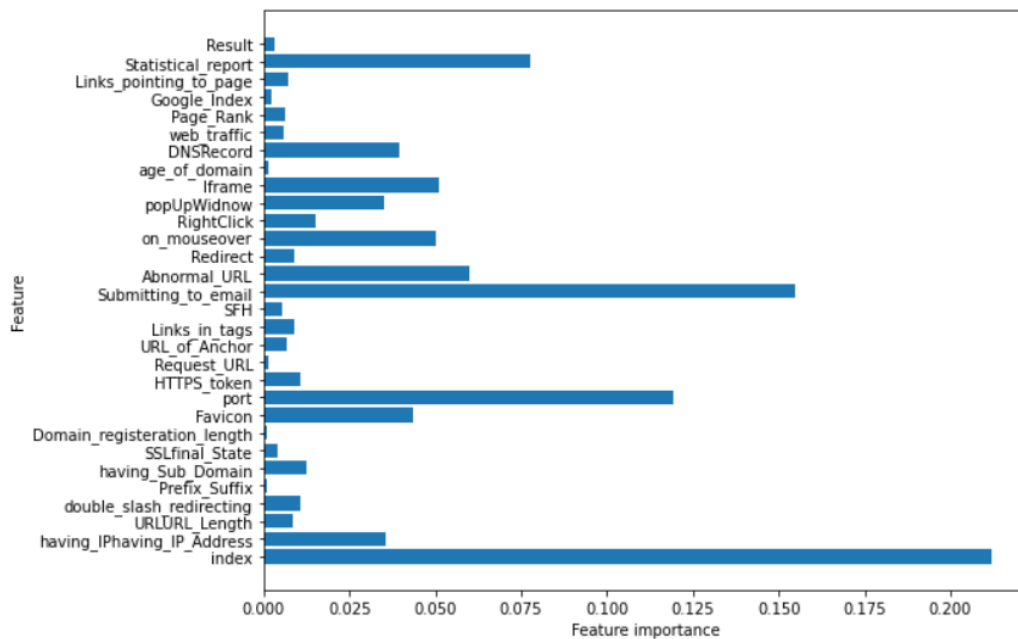
```
[41] #creating dataframe
results = pd.DataFrame({'ML Model': ML_Model,
                        'Train Accuracy': acc_train,
                        'Test Accuracy': acc_test})
results
```

	ML Model	Train Accuracy	Test Accuracy
0	Decision Tree	0.926	0.929
1	Random Forest	0.918	0.917
2	Multilayer Perceptrons	0.845	0.846
3	XGBoost	0.994	0.935

```
[42] #Sorting the dataframe on accuracy
results.sort_values(by=['Test Accuracy', 'Train Accuracy'], ascending=False)
```

	ML Model	Train Accuracy	Test Accuracy
3	XGBoost	0.994	0.935
0	Decision Tree	0.926	0.929
1	Random Forest	0.918	0.917
2	Multilayer Perceptrons	0.845	0.846

FEATURES:



3. TUNE THE MODEL: HYPER-PARAMETER TUNING.

```
In [58]: #HYPERPARAMETER TUNING
grid.fit(X_train, y_train)
```

```
Out[58]: * GridSearchCV
GridSearchCV(cv=5,
             estimator=GradientBoostingClassifier(learning_rate=0.7,
                                                    max_depth=4),
             param_grid={'max_features': array([1, 2, 3, 4, 5]),
                         'n_estimators': array([ 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130,
140, 150, 160, 170, 180, 190, 200])})

* estimator: GradientBoostingClassifier
GradientBoostingClassifier(learning_rate=0.7, max_depth=4)

* GradientBoostingClassifier
GradientBoostingClassifier(learning_rate=0.7, max_depth=4)
```

```
In [59]: print("The best parameters are %s with a score of %0.2f"
              % (grid.best_params_, grid.best_score_))

The best parameters are {'max_features': 5, 'n_estimators': 200} with a score of 0.97
```

10. ADVANTAGES & DISADVANTAGES

ADVANTAGES:

1. Improves the email gateways by providing phishing awareness and using trained algorithms.
2. Provides an ultimate solution in detecting whether a site is Phished or not by providing a friendly User Interface (U.I).
3. Since high efficiency and accuracy models are specifically picked and trained, the processing time is remarked less while providing high accuracy results.
4. Major previous turnback was in classification algorithm in which the respective code works. This area is focused and improved which provides better classification results among the competitive and existing algorithm and its combination.

DISADVANTAGES:

Although various machine learning algorithms have advanced, Phishers still manages to find a better alternative methodology to by-pass the security provided by the advanced algorithms.

Theocratically, it involves finitely deliverable data set and it can be easy to conclude. But in reality there are enormous number of phished websites, links and alternate ways that attackers find to develop and the fact is they exist in the real world entity. Practically, it is difficult to manipulate and analyse these many large data sets in to the algorithm. The characteristics also tend to change periodically making them dynamically modifying entity which can become quite complex to handle the scenario.

11.CONCLUSION

The final outcome and goal of this project is to provide a better classification algorithm by comparing and manipulating with the working structure of the existing top-class rated algorithms used in Machine Learning. This includes carefully picked up characteristics based upon F-level, Time Complexity, Accuracy and final deliverable result of the entire ideology involved. This helps the user to protect and create awareness from Phishing Attacks which is quite common and basic in terms but comparing would lead to a serious issues among the cyber-crime attacks. This simple schema enables the attackers to steal highly sensitive data such as private credentials which are solely an individual's entity. By providing friendly user interface to make accessible in providing a result of Phished or Legitimate is an ultimate challenge with evolving cyber-crime techniques in the adapting technology world. But machine learning is one such low-key are where it can be implemented as it is meant to learn the adaptations, thus proving the desired output.

12. FUTURE SCOPE

- Better code modifications can be done to improve reducing time complexity.
- Filtering methods in this case, Splitting data can be done in a manner it could segregate the necessary features from the data frame.
- Live tracking of Phished service offered by the attacker can be developed instead of manually relaying on the link from the user end.

13. APPENDIX

13.1 Source Code:

Model Building Source Code:

<https://github.com/IBM-EPBL/IBM-Project-25628-1659969123/blob/main/Project%20Development%20Phase/Sprint%202/Model-Building.ipynb>

13.2 GitHub and Project Demo Link:

GitHub Main Repository:

<https://github.com/IBM-EPBL/IBM-Project-25628-1659969123>

Project Demo Link:

<https://drive.google.com/drive/folders/1dGVcsSK54NdASTLV0Hc1uCkfZgDmHAKp?usp=sharing>